Problem Set 3

Fundamentals of Numerical Programming I

Leonard Storcks

January 12, 2026

Problem 1. Numerical derivatives by finite differencing

Consider a continuously differentiable function $f : \mathbb{R} \to \mathbb{R}$. Based on evaluations at points x_i with $x_{i+1} = x_i + \Delta x$, we would like to approximate the derivative $f_i' \equiv \frac{df}{dx}(x_i)$.

In the lecture, we have derived

- the first order forward difference formula

$$f_i' = \frac{f_{i+1} - f_i}{\Delta x} + \mathcal{O}(\Delta x) \tag{1}$$

- and the second order central difference formula

$$f_i' = \frac{f_{i+1} - f_{i-1}}{2\Delta x} + \mathcal{O}(\Delta x^2) \tag{2}$$

based on Taylor expansion.

(a) We would now like to derive a symmetric 4-th order finite difference formula to approximate f_i'. Determine the coefficients a and b in the symmetric Ansatz

$$f'(x) = a\,(f(x + \Delta x) - f(x - \Delta x)) + b\,(f(x + 2\Delta x) - 2f(x - \Delta x)) + \mathcal{O}(\Delta x^4) \tag{3}$$

to obtain a 4-th order finite difference approximation of $f'(x)$ by Taylor expansion.

(b) Implement Python functions to calculate

- first order forward differencing formula
- second order central differencing formula
- symmetric fourth order differencing formula

You may pass the following inputs:

- the function to be differentiated
- the evaluation point
- Δx

We will call the result from the implementation of the finite difference formula $\hat{f}_i'$ from now on.

(c) Consider the function $f(x) = L(L(L(x)))$ where L is the *logistic map* $L(x) = 4x(1-x)$.

- Find the analytical derivative of f by chain rule. *Hint:* You don't need to carry out the multiplications.
- For $x_0 = 0.2$ and $\Delta x = \text{np.logspace}(-15, -3, \text{num} = 200)$ calculate the error term $\text{err} = |\hat{f}_0' - f_0'|$ for all three finite difference formulas. Plot the error term over Δx in a double-logarithmic plot.

(d) Where is the round-off error, where the truncation error dominant? Explain the behavior you observe.

Problem 2. Newton interpolation and Runge's phenomenon

Consider the *Runge* function

$$f_A(x) = \frac{1}{1+25x^2} \tag{4}$$

as well as

$$f_B(x) = \sin(x) \tag{5}$$

and

- equidistant evaluation points

$$x_i = \frac{2i}{m} - 1, \quad i = 0, ..., m \tag{6}$$

- as well as *Chebyshev-nodes*

$$\tilde{x}_i = \cos\left(\frac{(2i+1)\pi}{2(m+1)}\right), \quad i = 0, \ldots, m \tag{7}$$

In this task, we will perform **Newton interpolation**

$$p(x) = \sum_{j=0}^{m} d_j n_j(x), \quad n_j(x) = \prod_{i=0}^{j-1}(x - x_i) \tag{8}$$

with the coefficients given by the finite differences

$$d_j = [x_0, ..., x_j]f \tag{9}$$

where for $j < i$ we recursively define

$$\begin{aligned} [x_j]\, f &= f_j, \\ [x_j, \ldots, x_i]\, f &= \frac{[x_{j+1}, \ldots, x_i]\, f - [x_j, \ldots, x_{i-1}]\, f}{x_i - x_j} \end{aligned} \tag{10}$$

(a) Implement Newton divided differences in Python.

(b) Implement the modified Horner scheme to evaluate $p(x)$ via

$$z_m = d_m, z_j = d_j + (x - x_j)z_{j+1} \quad \rightarrow \quad z_0 = p(x) \tag{11}$$

(c) For both f_A and f_B perform Newton interpolation for $m = 3, 9, 24, 48$ both with

equidistant and Chebyshev nodes. Plot the results. For plotting $p(x)$ use $m = 200$ evaluation points. Interpret your results.

Problem 3. *(Optional)* The power of finite differencing - solving the Kuramoto–Sivashinsky equation

The Kuramoto–Sivashinsky equation[1] is given by

$$\partial_t u + \alpha L \Delta u + \beta L^3 \Delta^2 u + \frac{\gamma L}{2u_s} |\underline{\nabla} u|^2 = 0 \tag{12}$$

where Δ is the Laplace and Δ^2 the biharmonic operator. $L = 1.0$ is the box size and α, β, γ are parameters, here

$$\alpha = 10^{-2}, \quad \beta = 10^{-6}, \quad \gamma = 10^{-2} \tag{13}$$

(physically all are in velocity units) and $u_s = 1.0$.

We consider a 2D setting, so

$$\Delta = \partial_x^2 + \partial_y^2 \tag{14}$$

$$\Delta^2 = (\partial_x^2 + \partial_y^2)(\partial_x^2 + \partial_y^2) = \partial_x^4 + \partial_y^4 + 2\partial_x^2 \partial_y^2 \tag{15}$$

We assume periodic boundary conditions and a box of size $L = 1$.

We discretize space to a grid with N^2 grid points

$$\begin{aligned} x_i &= L\frac{i}{N}, \quad i = 0, ..., N-1 \\ y_j &= L\frac{j}{N}, \quad j = 0, ..., N-1 \end{aligned} \tag{16}$$

such that the grid-spacing is $\Delta x = \frac{L}{N}$. Here we use $N = 128$.

Let us define the distance to the center of the box as

$$r_{ij} = \sqrt{\left(x_i - \frac{L}{2}\right)^2 + \left(y_j - \frac{L}{2}\right)^2} \tag{17}$$

The initial conditions are given by

[1] Here rather a physicists version of it.

$$u_{ij}(t=0) = \exp\left(-\eta \frac{r_{ij}}{L}\right), \quad \eta = 500 \tag{18}$$

Time is discretized into an iteration with N_t steps of size Δt with

$$t_{\text{final}} = 1.5L, \quad \Delta t = \frac{10^4}{L^3}\Delta x^4, \quad N_{\text{t}} = \lfloor \frac{t_{\text{final}}}{\Delta t} \rfloor \tag{19}$$

(a) Implement discrete Laplace, Biharmonic and gradient-squared operators based on the first and second derivative implementations given in Code-Snippet 1.

(b) Implement the right-hand side of the KS equation

$$\partial_t u = -\alpha L \Delta u - \beta L^3 \Delta^2 u - \frac{\gamma L}{2u_s} |\underline{\nabla} u|^2 \tag{20}$$

Use the discrete Laplace, Biharmonic and gradient-squared operators just implemented.

(c) Implement a time integration loop taking in an initial state $u(t_0)$ the time step Δt and the number of time steps N_s and returning the final state $u(t_0 + N_s \Delta t)$. A single time step is given by

$$u(t + \Delta t) \approx u(t) + \Delta t \, \partial_t u(t) \tag{21}$$

$\partial_t u(t)$ is given by the function you have just implemented.

(d) Run the simulation with the given initial conditions. Plot the final state using plt. imshow. The expected result is shown in Fig. 1

(e) Produce an animation of the state over time. Do not use every physical time step of your simulation as a frame but rather 150 frames equally spaced in time.

```python
import numpy as np

XAXIS = 0
YAXIS = 1

def first_deriv(u, grid_spacing, axis):
    return (np.roll(u, 1, axis=axis) - np.roll(u, -1,
    ↪  axis=axis)) / (2 * grid_spacing)

def second_deriv(u, grid_spacing, axis):
    return (np.roll(u, 1, axis=axis) - 2 * u + np.roll(u, -1,
    ↪  axis=axis)) / grid_spacing**2
```

Code-Snippet 1: Finite difference approximations of the first and second derivative along a specified axis of an array. By rolling the arrays, we implicitly assume periodic boundary conditions.

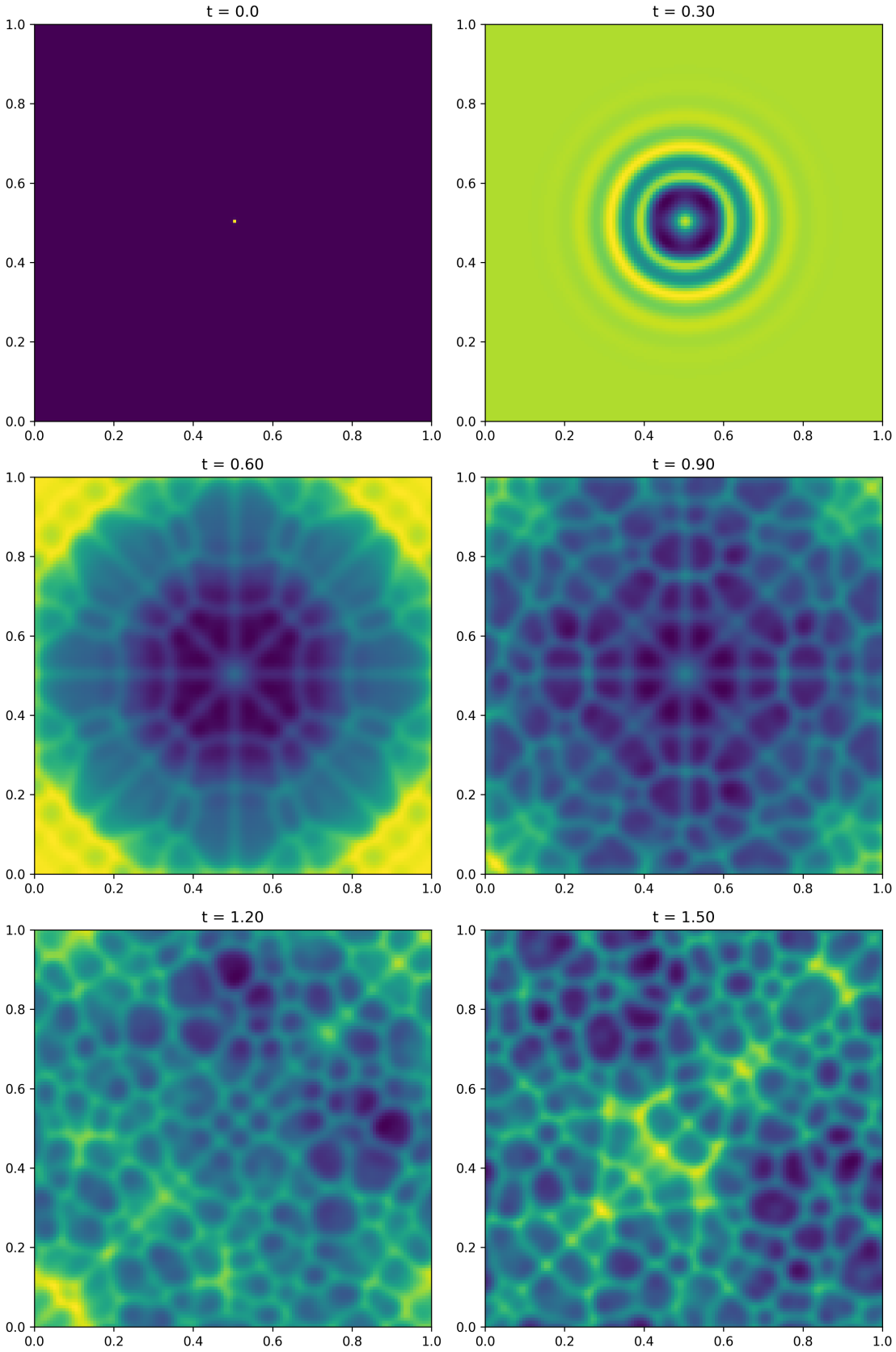

Figure 1: States in the time evolution of the Kuramoto–Sivashinsky equation.